

PGCP-AI Mock Assessment

Question 1: Custom Frequency Counter

Problem Statement

Implement a custom frequency counter without using Python's `collections.Counter` or any equivalent built-in frequency-counting utility.

Given a list of elements, compute and return the frequency of occurrence of each unique element.

Example

Input

```
items = ["AI", "ML", "AI", "Python", "ML", "AI"]
```

Output

```
{
    "AI": 3,
    "ML": 2,
    "Python": 1
}
```

Evaluation Criteria

- Correct frequency computation
 - Efficient use of dictionaries/hash maps
 - Time and space complexity analysis
-

Question 2: Active User Session Tracking

Problem Statement

You are provided with a sequence of user activity logs containing login and logout events.

Determine the set of users who remain actively logged in after processing all events.

Sample Input

```
logs = [  
    ("user1", "login"),  
    ("user2", "login"),  
    ("user1", "logout"),  
    ("user3", "login")  
]
```

Expected Output

```
["user2", "user3"]
```

Requirements

- Handle multiple login and logout events efficiently.
- Ignore invalid logout operations for users who are not logged in.
- Return the final list of active users in sorted order.

Evaluation Criteria

- Correct state management
 - Appropriate use of sets and dictionaries
 - Handling edge cases
-

Question 3: Mini TF-IDF Based Document Retrieval

Problem Statement

Develop a simplified TF-IDF (Term Frequency–Inverse Document Frequency) calculator for a collection of text documents.

Your implementation should:

1. Tokenize the documents.
2. Compute TF (Term Frequency).
3. Compute IDF (Inverse Document Frequency).
4. Calculate TF-IDF scores.
5. Identify the most important term(s) in each document based on TF-IDF score.

Sample Input

```
documents = [  
    "machine learning uses data",  
    "deep learning improves models",  
    "data science and machine learning"  
]
```

Expected Output

Display the highest TF-IDF scoring term for each document.

Evaluation Criteria

- Understanding of NLP fundamentals
 - Correct TF-IDF computation
 - Code modularity and scalability
-

Question 4: Design an LRU Cache

Problem Statement

Design and implement an LRU (Least Recently Used) Cache supporting the following operations:

```
get(key)
put(key, value)
```

Both operations must execute in **O(1)** average time complexity.

Example

```
cache = LRUCache(2)

cache.put(1, 100)
cache.put(2, 200)

cache.get(1)      # Returns 100

cache.put(3, 300) # Evicts key 2

cache.get(2)      # Returns -1
```

Requirements

- Implement cache eviction based on least recent usage.
- Support configurable cache capacity.
- Ensure constant-time access and update operations.

Evaluation Criteria

- Data structure selection
- Algorithmic efficiency
- Object-oriented design